

A Complete State Transition-Based Traffic Signal Control Using Deep Reinforcement Learning

Shangrui Liu
ECE Department
University of California at Riverside
Riverside, USA
sliu285@ucr.edu

Guoyuan Wu
CE-CERT and ECE Department
University of California at Riverside
Riverside, USA
gywu@cert.ucr.edu

Matthew Barth
CE-CERT and ECE Department
University of California at Riverside
Riverside, USA
barth@ece.ucr.edu

Abstract— Traffic signal control is a fundamental but challenging real-world problem that manages traffic at roadway intersections by adjusting signal timing and phases sequences. With the advances in emerging transportation technologies (e.g., Connected and Automated Vehicles, roadside sensing, drones), richer information (e.g., vehicle position, speed, type) has now become available in real-time, which can be utilized to reduce congestion. This paper introduces a deep reinforcement learning (DRL)-based traffic signal control (TSC) method for isolated intersections, where vehicles' positions and speeds (available via V2I, roadside sensing, or drone-based surveillance) are processed by a convolution neural network (CNN) and fed into the RL system as inputs. In addition, a complete state transition process using a dual-ring mechanism is introduced to enable flexible traffic signal control. Using a traffic simulator (SUMO: Simulation of Urban Mobility), the proposed algorithm is compared with both fixed-time and actuated TSC under different traffic conditions. Results show that the DRL-based strategy can reduce average delay and pollutant emissions by 34.7% and 18.5%, respectively.

Keywords—adaptive traffic signal control, deep reinforcement learning, state transition, emissions

I. INTRODUCTION

Our cities have been getting increasingly congested over the past several decades – primarily due to population growth and ongoing urbanization. This issue has received much attention from the researchers and practitioners who are proactively exploring various solutions, including how to effectively utilize existing traffic infrastructure to improve mobility within the limited resources. As an integral part of the contemporary traffic infrastructure, traffic signals manage traffic flows at urban intersections by adjusting signal timings and phase sequences. However, traditional traffic signal control (TSC) strategies such as fixed-time control and actuated control rely on minimal information for traffic management, thus resulting in degraded performance, especially when traffic demands

vary [1]. Adaptive signal control (ASC) strategies that leverage advanced detection to predict arrival patterns can provide better performance with limited flexibility [2]. Most of these strategies are rule-based and model-based.

The emergence of Connected and Automated Vehicles (CAVs) and Vehicle-to-everything (V2X) communications now provides a variety of opportunities to improve TSC in terms of mobility, efficiency, and environmental sustainability [3]. In addition, data-driven TSC strategies have been developed over the years due to significant improvement in information availability. However, to tap on the full potential of CAV-based TSC, high or even full penetration rates of connectivity or even automation is required [4]. At the same time, with recent successes in reinforcement learning (RL), there is an increasing interest in applying this technique to improving traffic signal control [5, 6]. RL can directly learn from the observed data without making strong assumptions about traffic patterns and human intervention. This is an attractive feature for the real-world implementation of TSC strategies. However, existing studies on RL-based TSC lack the consideration of complete flexibility of traffic signal transitions, thus capping the maximum benefits from RL techniques.

This study explores a complete state transition for traffic signal control at isolated intersections and develops a Deep Reinforcement Learning (DRL) strategy to improve system performance. The information used for DRL-based strategy may be obtained via V2X communications, roadside sensing, or even drone-based surveillance. The main contributions of the paper are:

- Developing a dual-ring controller with 65 traffic states, which provides complete flexibility in terms of state transition. This also gives the learning agent more freedom to explore the optimal policy from the environment.
- Developing a DRL model for the TSC at a single intersection, where a knowledge-assisted

decision-making algorithm is embedded to prevent frequent phase transitions, and a Deep Q-Network (DQN) algorithm is utilized to help the agent find the optimal policy.

- Validating the proposed algorithm by comparing with classical traffic signal control algorithms (both fixed-time and actuated) under scenarios of various traffic demands. Results of the simulation experiments confirm the superiority of the proposed algorithm in terms of average travel delay, emissions (e.g., CO_2 , CO , NO_x), and fuel economy.

The rest of this paper is organized as follows: Section II formulates the problem of TSC at an isolated intersection, followed by a detailed description of methodology in Section III. Section IV presents the simulation results in SUMO, and the last section concludes this study.

II. PROBLEM FORMULATION

We consider a deep reinforcement learning (DRL)-based traffic signal control problem. Figure 1 (inside the left block) shows an example intersection with multiple lanes on each approach. Vehicles entering the intersection are managed by traffic signals, including green, yellow, and red lights, which correspond to different movements of each approach. Traffic lights are grouped into several signal phases and operated in a compactable manner to guarantee safety at the intersection area.

Unlike many other RL-based TSC studies that assume phases sequence is predefined, this paper explores a much more flexible phase sequence based on the ring-barrier controller structure as shown in Figure 2. The set of all movements combinations at the intersection is denoted by M , i.e.,

$$M = \{[1,5], [1,6], [2,5], [2,6], [3,7], [3,8], [4,7], [4,8]\}$$

Besides, the minimum and maximum green times of phase $m \in M$, namely, $t_{m,gmin}$ and $t_{m,gmax}$ are set to use as the lower bound and upper bound for each phase duration. As for the ring-barrier controller, two rings include $\{1,2,3,4\}$ and $\{5,6,7,8\}$, respectively, which are two sets of self-conflicting movements. At every time step, the active movements need to be from the same side of barriers (Main Street or Side Street); for example, movement 1 and movement 6 can be activated simultaneously at the same time, while movement 1 and movement 7 cannot be activated simultaneously.

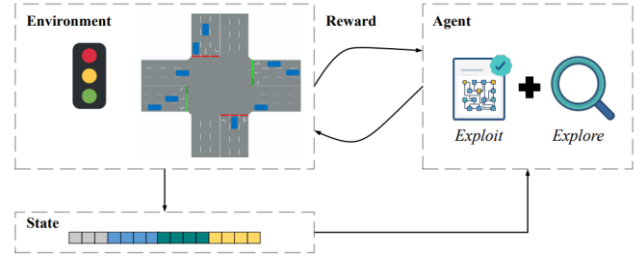


Fig. 1. Reinforcement learning based traffic signal control framework

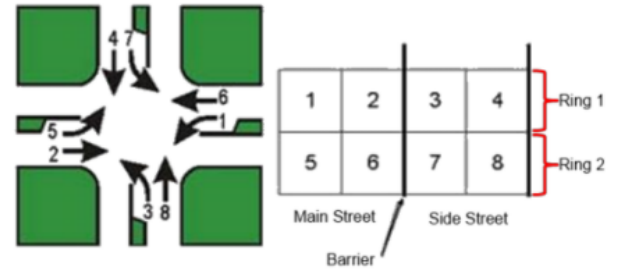


Fig. 2. Signal phase diagram in a ring-barrier controller [7]

This study aims to develop a deep reinforcement learning-based algorithm to flexibly control traffic lights at the target isolated intersection to achieve satisfactory system performance (e.g., throughput maximization, traffic flow rate balancing across different signal phases).

III. METHODOLOGY

This section presents: a) a DQN-based algorithm to learn the optimal control policy; and b) a knowledge-based algorithm to prevent frequent state transitions.

A. Model

This section uses the Markov Decision Process (MDP) framework to model the TSC problem in the RL context. The MDP is defined by a tuple $\langle S, A, P, R, \gamma \rangle$, where S is the state space, A is the action space, P is the state transition dynamic model, R is the reward function, and $\gamma \in [0, 1]$ is the discount factor. Thus, traffic signal control at signal intersections is formulated as an RL problem. An RL agent will learn the control strategy independently.

At a time step $t \in T$, the agent I receives the traffic state s^t from the environment. Given the state s^t , the agent takes an action a^t based on the policy π . The traffic environment transits from state s^t into a new state s^{t+1} according to the state transition dynamics $P(s^{t+1}|s^t, a^t)$, and the agent receives a reward r^{t+1} . The goal of the learning-based agent I is to find an optimal policy π^* that maximizes the expected cumulative rewards $E[\sum_t^T \gamma R(s^t)]$ from time step t onward.

1) State space S

The selection of state space is important since it determines agent decisions. For the traffic signal control problem, the states could be traffic information collected by roadside sensors (e.g., cameras) or V2X communications, including speed, position, and queue length. The proposed DRL-based model in this study selects vehicle speed, position, and traffic light state to compose the state space. A new state will be generated at the beginning of each time step.

All the lanes connecting to the intersection are discretized into a rectangular cell with a length c . The value of c is selected to be the average vehicle length plus minimum safety clearance to ensure no two vehicles fall into the same cell. Figure 3 demonstrates an example of the speed and position matrices along eastbound direction. Each cell contains a two-tuple to store the position and velocity, as shown in Figure 3 (b), (c). Regarding the position matrix (Figure 3 (b)), the value in each cell could be 0 and 1, depending on the corresponding cell's occupancy condition (with respect to Figure 3(a)). The position value will be 0 if no vehicle (or less than half of vehicle length) is present and 1 otherwise. The velocity matrix (Figure 3(c)) stores the vehicle speed (normalized by the speed limit) present in the associated cell. The sizes of position and speed matrices remain the same ($m \times n$), where m is the number of lanes from every approach ($|I|$) and $n = L/c$, where L is the length of target road segments approaching the intersection. Besides these two matrices, the third element in the state space is the traffic light state. It is a $1 \times |M|$ vector, where $|M|$ is the total number of phases at the intersection. The corresponding position in the traffic light state vector of the current active phase is set to 1, and other values in the vector are set to 0.

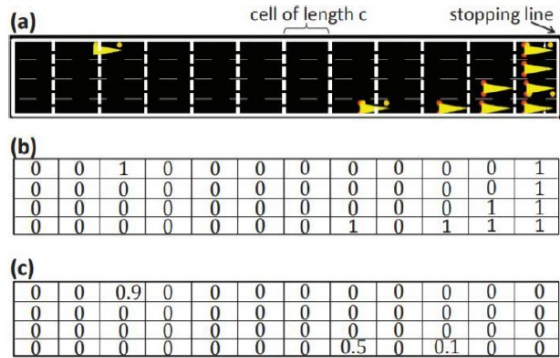


Fig. 3. Demonstration of traffic state conversion into state space

2) Action space A

In this study, the phase sequence is not predefined, and the controller determines which phase should be up at each time step (with safety constraints) to maximize the reward. The action space is defined as a $1 \times |M|$ vector. The agent

I selects an action among all the phases at each time step, and the corresponding position in the action matrix is set to 1 while other values in the matrix are set to 0. To guarantee the safety performance at the intersection, the yellow light state (with duration t_{yellow}) and all-red state (with duration t_{allred}) are triggered when the traffic signal transits from one green phase to another green phase. In addition, minimum green time $t_{m,gmin}$ and maximum green time $t_{m,gmax}$ are set for each phase according to the requirements of authorities. These constraints will also govern the behaviors of RL agents.

3) Transition dynamics P

The state transition dynamics P is a critical element in the RL problem. It shows how the environment reacts to the action taken by the RL agent. Specific to the TSC problem, the state transition shows how the traffic light transfers from one state to another. In this study, the traffic light transition is controlled by the scheme as shown in Figure 4, which can provide maximal flexibility for the traffic light transitions. In most previous studies, any two non-conflict movements need to be activated and inactivated together, limiting the agent's capability to learn the optimal policy. State transitions in this study do not necessarily have a coupling of movements on different rings, e.g., movement 1 and movement 6, or movement 2 and 6. Any movement can be activated individually or with any non-conflict movements from the same side of barriers (but different rings). As shown in Figure 4(a), there are in total 33 possible traffic signal states, which are represented as circles. This state transition diagram is considered as an extension of our previous work [8]. Green circles represent states where two green movements are activated; yellow circles describe states where at least one movement is yellow, but the other is not red; and red circles denote states where at least one phase is red.

The arrows show how the state transits from one to another. It can be observed that the transitions defined herein are more flexible than those in other studies. Considering the figure legibility, some transitions are not shown in Figure 4(a), e.g., from 1Y/6R to 1Y/5G and from 1Y/6R to 1R5G. Figure 4(b) is the state transition diagram for both Main Street and Minor Street, where the All-Red state is needed when the traffic light traverses cross from one side of the barrier to the other.

4) Reward function R

The reward function determines not only the convergence speed of RL algorithms but also the system performance. The average delay, average waiting timing, and queue length are often considered as the key objective functions for traffic signal control. In this study, four different reward functions are evaluated:

- Lane level average queue length related (AQ)

$$r_t^2 = \left| \sum_{i=\text{lane id}} q_t^{a,i} / |i| - \sum_{j=\text{lane id}} q_t^{b,j} / |j| \right| \quad (1)$$

- Waiting time related (WT)

$$r_t^3 = r_a - r_b \quad (2)$$

- Combination of r_t^2 and r_t^3 (WQ)

$$r_t^4 = w_1 * r_t^2 + w_2 * r_t^3 \quad \text{and} \quad w_1 + w_2 = 1 \quad (3)$$

- Lane level max queue length related (MQ)

$$r_t^1 = \left| \max_{i=\text{lane id}} \{q_t^{a,i}\} - \max_{j=\text{lane id}} \{q_t^{b,j}\} \right| \quad (4)$$

where $q_t^{a,i}$ and $q_t^{b,j}$ are the number of queued vehicles at time t for phases a and b , respectively; r_a and r_b are the cumulated waiting times for vehicles corresponding to phases a and b .

B. Algorithms

Two major algorithms are developed for the proposed DRL-based strategy. The first is a model-free DRL algorithm that uses Q-learning to find the optimal policy. The second is a knowledge-based decision algorithm that helps the RL agent to learn the optimal policy while preventing frequent state transitions.

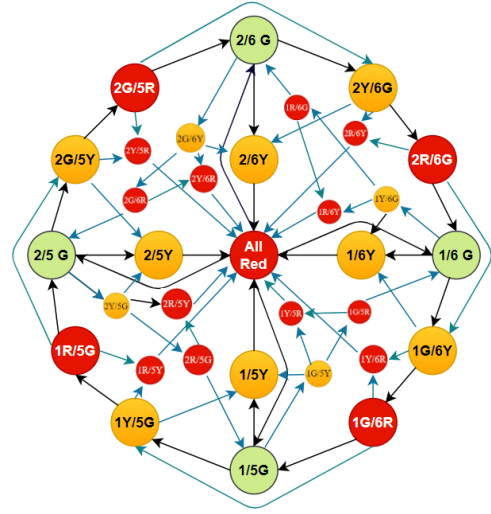
1) Deep Q-learning traffic signal control

A model free DRL algorithm, called Q-learning, is applied in the study to guide traffic signal controller to select control policy. The Q-learning uses a Q-value function $Q(s, a)$ to evaluate how good an action is in each state. The Q-value function is updated by the following equation:

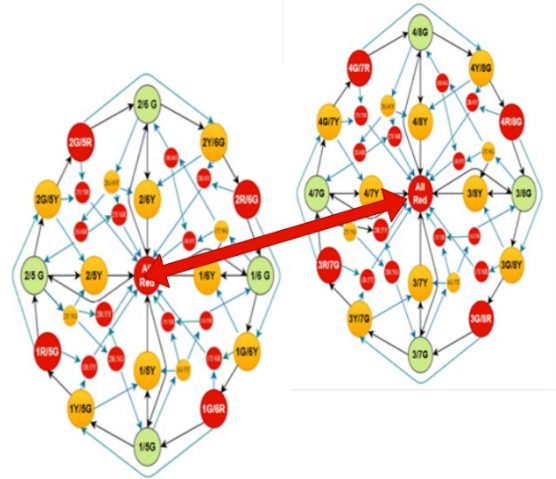
$$Q(s, a) \leftarrow Q(s, a) + \alpha [r_t + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (5)$$

where s and s' are the current and next state; a and a' are the current and next action; r_t is the immediate reward based on which type of reward function is chosen; $\max_{a'} Q(s', a')$ is the maximum possible reward for the next state; $\alpha \in [0, 1]$ is the learning rate, and $\gamma \in [0, 1]$ is the discount factor.

One challenge for using Q-learning to solve the DRL-TSC problem is that the traffic environment is extremely complex and highly dynamic. It is intractable to store all the state-action pairs into a Q-table, and almost impossible to trace all the possible traffic states, even for an isolated intersection. To solve this non-stationary learning problem, a Deep Q-Network (DQN) is introduced as a function approximator to estimate Q-value.



(a) Main street



(b) Main street and minor street including the connection state "All Red"

Fig. 4. State transition diagram

The architecture of DQN is shown in Figure 5. The network input is observed states, including position matrix (P), speed matrix (V), and traffic light states vector (L). Specifically, in this study, the first layer of the convolution neural network (CNN) has 16 filters of size 4×4 with stride of 2, and it applies ReLU as the activation function. The second layer has 32 filters of size 2×2 with stride of 1, and also uses ReLU as the activation function. The third and fourth layers are fully connected (FC) layers of sizes 128 and 64, respectively. The final layer is the linear layer outputting Q values corresponding to every possible action that the agent takes.

The model was first initialized by random weights to train the neural network. At the beginning of every step, the RL agent observes the current state s_t , and performs action a_t with the highest cumulative future reward. After performing the action, the agent receives reward r_t^i and updates the next state s_{t+1} from the environment. The agent stores this experience $(s_t, a_t, r_t^i, s_{t+1})$ in memory. DNN is trained by extracting training examples of the type (s_t, a_t) from the memory. After collecting the training data, the agent learns features θ by training the DNN network to minimize the following Mean Squared Error (MSE):

$$MSE(\theta) = \frac{1}{m} \sum_{t=1}^m \left\{ \left(r_t^i + \gamma \max_a Q(s_{t+1}, a_{t+1}; \theta_{t+1}) \right) - Q(s_t, a_t; \theta) \right\}^2 \quad (6)$$

where m is the size of the input data set. Since m is large, the computational cost to calculate $MSE(\theta)$ is considerable. A stochastic gradient descent algorithm RMSProp is applied with a minibatch of size 32. More detailed procedures are described in **Algorithm 1**.

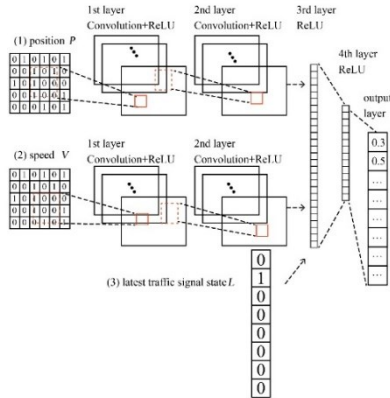


Fig. 5. The architecture of proposed Deep Q-Network (DQN)

2) Knowledge-based decision

The knowledge-based decision algorithm is proposed to prevent frequent state transitions. Since a flexible structure is applied in this study, the state space is much larger than previous work. This may also cause frequent state transitions, which is inefficient for real-time traffic management. Therefore, a simple rule-based algorithm is developed to guarantee that the state would not transit to the next if it lasts for less than some threshold (e.g., minimum green time).

An indicator is introduced to supervise the agent action, which is set to 1 if there are still queues in current activated phase; otherwise, set to 0, as shown in Equation (7):

$$m = \begin{cases} a_t, & \text{if } f = 0 \\ m_{t-1}, & \text{if } f = 1 \end{cases} \quad (7)$$

where m is the phase executed at the current time step; a_t is the action suggested by the RL agent; and m_{t-1} is the active phase at the last time step. If f is set to be true, then the action suggested by the agent would be denied; and the same active phase remains; otherwise, the state transits to the one calculated by the RL agent. **Algorithm 2** summarizes the proposed *knowledge-based decision algorithm*, and Figure 6 illustrates the entire system flowchart.

Algorithm 1: DQN for Flexible Dual Ring Controller

Input: Mini-batch size B , learning rate α , discount factor γ , exploration probability ϵ , exploration decent rate k , reward type i
Output: optimal Q-value Vector
1: Initialize replay memory D , random weight $\emptyset$
2: **for** $episode=1$ to E **do**
3: Receive initial state s_t
4: **while** $t < T$ **do**
5: **if** probability ϵ
6: select random action a_t
7: **else** $a_t = \max_a Q(s_t, a_t; \theta)$
8: **if** frequent change flag = 0:
9: Execute a_t
10: Receive reward r_t^i , new state s_{t+1}
11: Store transition $(s_t, a_t, r_t^i, s_{t+1})$ in D
12: $t \leftarrow t + 1, \epsilon \leftarrow \epsilon * k$
13: Sample random mini-batch of experience
14: $y = r_t^i + \gamma \max_a Q(s_{t+1}, a_{t+1}; \theta_{t+1})$
15: performance a gradient descent
16: update Q

Algorithm 2: Knowledge-based Decision

Input: last time step active lane level queue vehicle id list I_{pre} , last time step active phase m_{pre}
Output: Frequent Change flag f
1: Initialize Queue vehicle id list I_{cur}
2: **for** $lane$ in m_{pre}
3: store veh to I_{cur}
4: **if** I_{cur} in I_{pre}
5: $f \leftarrow 1$ & reject a_t
6: **else**
7: $f \leftarrow 0$ & execute a_t
8: $I_{pre} \leftarrow$ get vehicle id list corresponding to a_t
9: **return** f

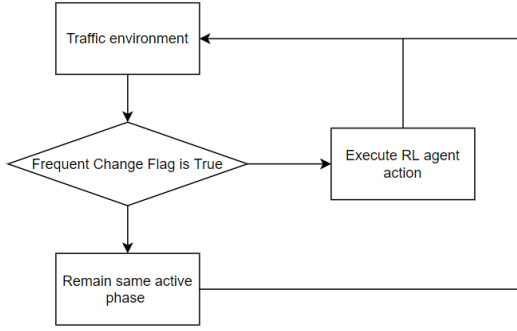


Fig. 6. System flowchart

IV. SIMULATION AND RESULTS

A. Simulation Environment

The experiments are conducted in a micro-simulation environment using the Simulation of Urban Mobility (SUMO) platform [9], which provides a user-friendly Graphical User Interface (GUI) and an Application Programming Interface (API) – TraCI, allowing users to design road networks and external control strategies. We use Python scripts to code the DRL controller. Real-time traffic states are obtained, the RL agent is trained, and traffic lights are controlled using TraCI. The dynamics of vehicles in SUMO are built by a car-following model that considers drivers' physical and psychological factors. Given the decision of traffic light, vehicles can adjust their trajectories. A real-world intersection (University Ave. and Chicago Ave. in Riverside, California) scenario is imported from OpenStreetMap [10] and slightly modified for consistency (see Figure 7). There are five lanes from each approach, including 1 shared lane, 1 dedicated left-turn lane, 2 dedicated through lanes, and 1 dedicated right-turn lane. Minimum and maximum green times ($t_{m,gmin}$ and $t_{m,gmax}$) are set as 10s and 40s, respectively; amber time and all-red clearance duration are set to be 5s and 1s, respectively.

Two types of traffic demands are introduced to test the robustness of the proposed DRL controller: the first one is a typical weekday afternoon peak hour (4:30 – 5:30 pm on June 2nd, 2016) traffic surveyed by the City of Riverside. There are in total 3,855 vehicles passing through intersections within this hour, and the traffic demands vary every 15 minutes. In the second type, we keep the total number of vehicles unchanged but further disturb the traffic demands for further disproportion across different 15-minute periods (see Table I).

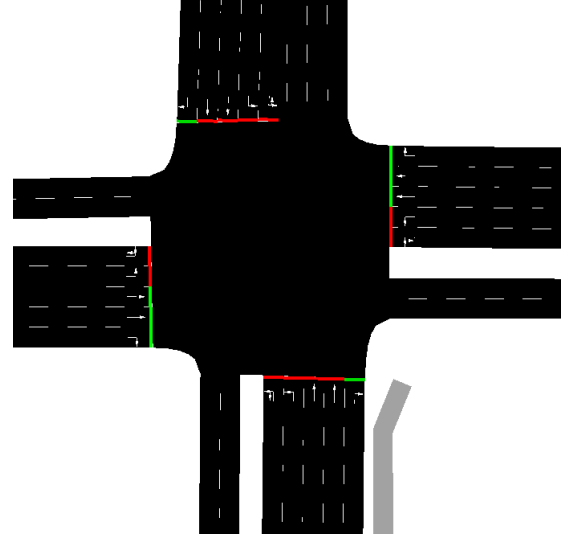


Fig. 7. The test intersection (University @ Chicago in Riverside, CA)

TABLE I. PROPORTIONS FOR TWO TYPES OF TRAFFIC DEMANDS

Type	Time Period			
	4:30 - 4:45	4:45 - 5:00	5:00 - 5:15	5:15 - 5:30
I	24.0%	27.1%	24.1%	24.8%
II	35.0%	15.0%	30.0%	20.0%

B. Control Strategies for Comparison

In this study, the proposed DRL-TSC is compared with two other types of traditional traffic signal control methods, i.e., fixed-time control and actuated control.

1) Fixed-time traffic signal control (FTTSC)

For FTTSC, the real-world implemented signal timing plan (background table) is used, which is supposed to be optimized based on the real-world traffic flows.

2) Actuated traffic signal control (ATSC)

For ATSC, the signal controller responds to real-time traffic flows by changing the duration of each phase but not the phase sequence. Loop sensors are modeled in the same settings as in the real world. Also, the real-world time table is adopted from the help of City traffic engineers.

3) DQN-based traffic signal control (DTSC)

The Deep Q-Network (DQN) architecture of the proposed controller has been described in previous section. The loop sensor is installed 100m from the stop line. The sizes of the position matrix (P) and speed matrix (V) are both 20×12 , since there are 20 lanes approaching the intersection. Each lane is discretized into 12 cells, with the length of each cell c being set as 7m. The outputting Q values vector has the size of 1×8 , since the total number of

possible phases $|M|$ is 8. For fair comparison, each phase's $t_{m,gmin}$ and $t_{m,gmax}$ in DTSC are set to be the same as ATSC. Table II summarizes the parameters of DRL model.

TABLE II. PARAMETERS OF DRL MODEL

Parameter	Value
Episodes E	300
Time step interval Δ	5s
Mini-batch Size B	32
Learning rate α	0.0002
Discount factor γ	0.95
Initial exploration rate ε	0.99
Minimize exploration rate ε_{min}	0.01
Exploration decay rate k	0.99

C. System Performance

In this section, system performance of the proposed DRL-based controller is evaluated in terms of the average travel delay and pollutant emissions such as CO_2 , CO , NO_x .

1) Average delay

Figure 8 and Table III present the average delay per vehicle for the six TLC methods (including FTTSC, ATSC and other four DTSC with different reward functions as shown in Eq. (1) – (4)), and the relative improvement with respect to FTTSC. As observed from the figure and table, all the DRL-based TSC strategies outperform the traditional TSC ones (i.e., FTTSC and ATSC). Among all DRL-based models, the one applying max queue length as its reward function shows the best performance. The average delay per vehicle under two traffic demands are 26.07s and 33.87s, with reductions of 23.80% and 34.70%, respectively. In addition, DRL-based TSC models are more robust to the disturbed traffic demands. For the FTTSC, the average travel delay is increased by 40.2% when the traffic demand is disturbed, while the DRL-MQ (max queue length) method only gets increased by 18.4%.

2) Total Vehicle Emissions

In this part, total vehicle emissions are evaluated, including CO_2 , CO , NO_x , and Table IV presents the results for different TSC methods under the real-world traffic demand. Similar to the average travel delay, the DRL-based TSC models consistently perform better than the other two traditional TSC methods. Among all DRL-based methods, the one applying max queue length as the reward function again outperforms others. The emissions of CO_2 , CO , NO_x

are 14.7%, 18.5%, and 13.5% less than FTTSC, respectively, for the scenario with real-world traffic demand.

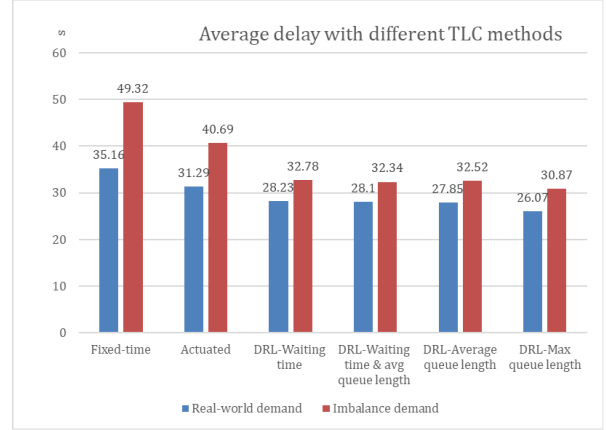


Fig. 8. Average travel delay of different TSC methods

TABLE III. AVERAGE TRAVEL DELAY AND IMPROVEMENT

TSC Strategies	Traffic Demand			
	Real-world		Disturbed	
	Abs.	Rel. (%)	Abs.	Rel. (%)
FTTSC	35.16	-	47.32	-
ATSC	31.29	11.0	40.69	14.0
DTSC-WT	28.23	19.7	32.78	30.7
DTSC-WQ	28.10	20.0	32.34	31.5
DTSC-AQ	27.85	20.7	32.52	31.3
DTSC-MQ	26.07	23.8	30.87	34.7

V. CONCLUSIONS AND FUTURE WORK

This study proposes deep reinforcement learning (DRL)-based traffic signal control (TSC) strategies for an isolated intersection, considering a complete state transition with high flexibility. Different reward functions, including average queue length (AQ), waiting time (WT), weighted combination of waiting time and average queue length (WQ), and maximum queue length (MQ), are investigated for DRL-based TSC methods. It turns out that DRL-MQ consistently outperforms other DRL-based TSC strategies, and provides much better system performance in terms of average travel delay and pollutant emissions than traditional control strategies, i.e., fixed-time and actuated controls.

As future steps, we plan to: 1) further evaluate the system performance of the proposed strategies in field operational tests; 2) investigate the applicability in a more complicated scenario (e.g., multiple modes); 3) explore more advanced machine learning techniques; and 4) extend

this effort to a signalized corridor or even a signalized grid network.

TABLE IV. TOTAL VEHICLE EMISSIONS FOR DIFFERENT TSC METHODS UNDER REAL-WORLD TRAFFIC DEMAND

TSC Strategies	Pollutant Emissions					
	<i>CO₂ (g/mi)</i>		<i>CO (g/mi)</i>		<i>NOx (g/mi)</i>	
	<i>Abs.</i>	<i>Rel.^a</i>	<i>Abs.</i>	<i>Rel.</i>	<i>Abs.</i>	<i>Rel.</i>
FTTSC	417.7	-	2.86	-	478.5	-
ATSC	375.7	10.1	2.54	11.2	443.6	7.3
DTSC-WT	356.1	14.7	2.37	17.1	427.0	10.7
DTSC-WQ	362.5	13.2	2.46	14.0	438.7	8.3
DTSC-AQ	358.4	14.2	2.35	17.8	421.8	11.8
DTSC-MQ	354.1	14.7	2.27	18.5	414.0	13.5

^a In percentage (%)

REFERENCES

- [1] H. Wei, et al., "A survey on traffic signal control methods," arXiv:1904.08117, 2019
- [2] A. Stevanovic, "Adaptive traffic control systems: domestic and foreign state of practice," Transportation Research Board, Vol. 17, 2010, p. 18.
- [3] Y. Feng, et al., "A real-time adaptive signal control in a connected vehicle environment," Transportation Research Part C: Emerging Technologies, Vol. 55, 2015, pp. 460–473.
- [4] K. Dresner and P. Stone, "A multiagent approach to autonomous intersection management," Journal of Artificial Intelligence Research, Vol. 31, 2008, pp. 591–656.
- [5] F. Rasheed, et al., "Deep reinforcement learning for traffic signal control: a review," IEEE Access, 2020, pp. 208016–208044, DOI: 10.1109/ACCESS.2020.3034141.
- [6] S. Park, et al., "Deep Q-network-based traffic signal control models," PLoS One, 2021, 16(9): e0256405. Published online 2021 Sep 2. DOI: 10.1371/journal.pone.0256405.
- [7] U.S. Department of Transportation Federal Highway Administration, "Signal timing on a shoestring," Washington, DC, 2005.
- [8] D. Kari, G. Wu, M. Barth, "Eco-friendly freight signal priority using connected vehicle technology: A multi-agent systems approach", 2014 IEEE Intelligent Vehicles Symposium, June, Dearborn MI, DOI: 10.1109/IVS.2014.6856511.
- [9] Simulation of urban mobility (SUMO), <https://www.eclipse.org/sumo/>. Accessible on October 20th, 2021
- [10] OpenStreetMap, <https://www.openstreetmap.org/>, Accesible on October